



ANALIZA TECHNICZNA · KWIECIEŃ 2026

Workflows v2

Zgodność z n8n

Jak działa obecny system workflow w Sembot, co oznacza „zgodność z n8n” i co konkretnie trzeba zmienić, żeby przejść do v2.

TEMPLATEK DO ANALIZY

38

OBSZARÓW ZMIAN

6

SKALA MIGRACJI

Duża

1 Czym jest workflow? Wyjaśnienie dla laika

Analogia do taśmy produkcyjnej

Wyobraź sobie workflow jak **automatyczną linię produkcyjną w fabryce**. Surowiec (dane) wchodzi jednym końcem, przechodzi przez kolejne stanowiska (węzły), na każdym coś się z nim dzieje (filtrowanie, przeliczanie, sprawdzanie warunków), a na końcu wyjeżdża gotowy produkt (np. raport, zadanie, powiadomienie). Cała linia uruchamia się sama, według harmonogramu — bez potrzeby, żeby ktoś stał przy każdej maszynie.

W Sembot workflow to **zestaw automatycznych zadań**, które uruchamiają się samoczynnie — na przykład co poniedziałek, sprawdzają dane z Google Ads, porównują wyniki i tworzą alert dla managera, gdy coś pójdzie nie tak. Użytkownik konfiguruje workflow raz, a potem system sam robi to za niego.

38

GOTOWYCH SZABLONÓW
W OBECNYM SYSTEMIE

5

TYPÓW WĘZŁÓW
(TRIGGER, GETTER, FILTER...)

6+

INTEGRACJI ZEWNĘTRZNYCH
(GOOGLE ADS, MICROSOFT...)

2 Jak działa obecny system workflow w Sembot

Obecny system workflow to **własne rozwiązanie Sembot** — zaprojektowane i zbudowane przez nasz zespół od podstaw. Każdy workflow jest opisany jako plik JSON (tekstowy zapis struktury danych), który przechowuje informacje o tym, co ma robić, kiedy i w jakiej kolejności.

Schemat działania — przykład alertu ROAS



Typy węzłów w obecnym systemie

Typ węzła	Co robi	Przykład
 Trigger	Uruchamia cały workflow — według harmonogramu lub zdarzenia	„Co poniedziałek o 8:00”
 Getter	Pobiera dane z zewnętrznych źródeł	Pobierz kampanie z Google Ads
 Flow Control	Steruje przepływem: filtruje dane lub iteruje po liście	Pokaż tylko kampanie z ROAS < 3

Typ węzła	Co robi	Przykład
 AI Agent	Przekazuje dane do modelu AI i zbiera odpowiedź	Przeanalizuj luki w słowach kluczowych
 Action	Wykonuje akcję końcową — tworzy zadanie, wysyła powiadomienie	Utwórz zadanie z alertem



Struktura pliku workflow (uproszczona)

Każdy szablon workflow to plik z dwoma sekcjami: **meta** — informacje wyświetlane użytkownikowi (nazwa, kategoria, ikona, korzyści) oraz **data** — właściwa definicja: lista węzłów (nodes), gdzie każdy wie, kto jest jego poprzednikiem (przez `parent_id`). Węzły są połączone liniowo — jeden po drugim.

Czym jest n8n i co oznacza „zgodność z n8n”

Czym jest n8n?

n8n (wymawiane: *nodemation*) to popularna, open-source'owa platforma do budowania automatyzacji i workflow. Działa jak „Zapier dla programistów” — pozwala łączyć różne serwisy i automatyzować procesy za pomocą wizualnego edytora. Ma setki gotowych integracji (Google, Microsoft, Slack, bazy danych...) i własny standardowy format opisu workflow w JSON.

? Co dokładnie oznacza „v2 workflows zgodne z n8n”?

Szef IT podał krótką informację — „workflows zgodne z n8n”. Może to oznaczać dwa scenariusze:

Opcja A — Kompatybilność formatu

Nasze szablony workflow zmieniają format JSON tak, żeby można je było importować i eksportować do/z n8n. Sembot wciąż może mieć własny silnik wykonania.

Opcja B — n8n jako silnik

Rezygnujemy z własnego executora workflow i używamy n8n jako backendu wykonującego workflow. Większa zmiana — dwa razy większy zakres pracy.



Rekomendacja przed planowaniem

Przed przystąpieniem do jakiegokolwiek migracji należy doprecyzować u szefa IT, który scenariusz jest docelowy. Różnica w nakładzie pracy jest bardzo duża. Poniższa analiza dotyczy **Opcji A — kompatybilności formatu**, czyli przepisania szablonów na schemat JSON n8n.

Co jest inne — porównanie formatów

Poniżej porównanie konkretnych elementów między obecnym formatem Sembot a formatem n8n. Każda różnica to obszar, który wymaga zmiany przy migracji do v2.

Element	Sembot dziś (v1)	n8n (v2)	Status
Identyfikatory węzłów	Liczby całkowite: 1, 2, 3...	UUID: "a1b2c3d4-0000-..."	Zmiana
Połączenia między węzłami	Pole parent_id w każdym węźle — liniowy łańcuch	Osobny obiekt connections — obsługuje rozgałęzienia i równoległe ścieżki	Zmiana
Typy węzłów	Własne nazwy: trigger, getter, action, flow_control, ai_agent	Pakietowe nazwy: n8n-nodes-base.scheduleTrigger, n8n-nodes-base.httpRequest	Zmiana

Element	Sembot dziś (v1)	n8n (v2)	Status
Wyrażenia dynamiczne	<pre>{3.campaignId} {NOW().subtractNumberOfDays(7)} {divide({4.x},{4.y})}</pre>	<pre>{{ \$node["Get"].json.campaignId }} {{ \$now.minus(7,'days').format('...') }} {{ \$json.x / \$json.y }}</pre>	Zmiana
Zmienne globalne (globals)	<pre>globals: { roas_target: { value: 3.0, type: "number" } }</pre>	Zmienne na poziomie workspace albo węzeł „Set” na początku workflow	Zmiana
Pozycja węzłów na canvas	Brak — kolejność przez pole index	Wymagane współrzędne: position: [x, y]	Dodać
Metadane szablonu (meta)	Zewnętrzna sekcja meta z: slug, category, icon, benefits, visible	Minimalne meta — templates n8n mają osobną bazę szablonów	Zachować
Wersjonowanie węzłów	Brak	Każdy węzeł ma typeVersion (np. 1.1)	Dodać

Poniżej lista konkretnych zmian, które muszą zajść w 38 szablonach workflow, żeby były zgodne z formatem n8n. Każdy element jest opisany: co jest teraz, co musi być po zmianie, i ile to wymaga pracy.

Krytyczne

1. Przepisanie wyrażeń dynamicznych — największa robota

Teraz: Własna składnia Sembot z nawiasami klamrowymi i funkcjami PHP-like. Przykład:

`{NOW().subtractNumberOfDays(30).formatDateTime('Y-m-d')}` lub odwołanie do danych z węzła:
`{3.campaignId}, {divide({4.x},{4.y})}`

Po zmianie: Składnia n8n z podwójnymi nawiasami i JavaScript. Przykład: `{{ $now.minus(30,'days').format('yyyy-MM-dd') }}` lub `{{ $node["GetCampaigns"].json.campaignId }}`, `{{ $json.x / $json.y }}`

SEMBOT DZIŚ (V1)

```
// data range
"{NOW().subtractNumberOfDays(7)
  .formatDateTime('Y-m-d')}"

// odwołanie do węzła #3
"{3.campaignId}"

// obliczenie
"{divide({4.conversions_value},
  {4.cost})}"
```

N8N (V2)

```
// data range
"{{ $now.minus(7, 'days')
  .format('yyyy-MM-dd') }}"

// odwołanie do węzła po nazwie
"{{ $node['GetCampaigns']
  .json.campaignId }}"

// obliczenie
"{{ $node['GetMetrics'].json
  .conversions_value /
  $node['GetMetrics'].json.cost }}"
```

Krytyczne

2. Zmiana sposobu łączenia węzłów (parent_id → connections)

Teraz: Każdy węzeł ma pole `parent_id` wskazujące poprzednika. Tworzy to prosty, liniowy łańcuch: węzeł 1 → 2 → 3 → 4.

Po zmianie: Połączenia opisane są w osobnym obiekcie `connections` poza węzłami. Każdy węzeł ma nazwę, a `connections` mapuje: „z węzła X idź do węzła Y”. To pozwala też na rozgałęzienia (jeden węzeł → dwa różne).

SEMBOT DZIŚ (V1)

```
"nodes": [
  {
    "id": 1,
    "parent_id": null,
    "type": "trigger"
  },
  {
    "id": 2,
    "parent_id": 1, — połączenie
    "type": "getter"
  }
]
```

N8N (V2)

```
"nodes": [
  { "name": "Schedule", ... },
  { "name": "GetData", ... }
],
"connections": { — osobny obiekt
  "Schedule": {
    "main": [[{
      "node": "GetData",
      "type": "main"
    }]]
  }
}
```

Krytyczne

3. Mapowanie typów węzłów na odpowiedniki n8n

Krytyczne

Teraz: Własne, generyczne typy węzłów: `trigger`, `getter`, `action`, `flow_control`, `ai_agent`. Podtyp określa pole `payload.type`.

Po zmianie: Każdy węzeł musi mieć konkretny typ n8n — np. `n8n-nodes-base.scheduleTrigger`, `n8n-nodes-base.filter`, `@n8n/n8n-nodes-langchain.agent`. Dla integracji Sembot-specyficznych (Google Ads, Phrase Monitor) potrzebne będą **własne custom nodes** zarejestrowane w n8n.

Sembot typ (v1)	Podtyp payload	Odpowiednik n8n (v2)
trigger / schedule	schedule	<code>n8n-nodes-base.scheduleTrigger</code>
flow_control / filter	filter	<code>n8n-nodes-base.filter</code>
flow_control / iterator	iterator	<code>n8n-nodes-base.splitInBatches</code>
ai_agent	ai_agent	<code>@n8n/n8n-nodes-langchain.agent</code>
getter	<code>get_google_ads_*</code>	Custom node: <code>sembot-nodes.googleAds</code> ⚠
getter	<code>get_phrase_monitor_*</code>	Custom node: <code>sembot-nodes.phraseMonitor</code> ⚠
action	<code>create_task</code>	Custom node: <code>sembot-nodes.createTask</code> ⚠



Węzły oznaczone ⚠ wymagają stworzenia od podstaw

Integracje specyficzne dla Sembot (Google Ads, Phrase Monitor, tworzenie zadań) nie mają odpowiedników w standardowej bibliotece n8n. Trzeba zbudować własne „custom nodes” — to oddzielny projekt deweloperski.

Wysokie

4. Zmiana identyfikatorów węzłów z liczb na UUID

Teraz: Węzły mają proste numeryczne ID: 1, 2, 3, 4, 5... Używane zarówno do identyfikacji jak i w wyrażeniach: `{3.campaignId}`

Po zmianie: n8n wymaga UUID w formacie: "a1b2c3d4-e5f6-7890-abcd-ef1234567890" Wyrażenia odwołują się do węzłów przez **nazwę**, nie ID — więc numery w wyrażeniach też znikną (patrz zmiana #1).

Wysokie

5. Zmiana obsługi zmiennych globalnych (globals)

Teraz: Workflow ma sekcję `globals` z nazwanymi zmiennymi dla konkretnego workflow, np: `roas_target: { value: 3.0, type: "number" }`. Używane w wyrażeniach przez `{globals.roas_target}`

Po zmianie: n8n nie ma bezpośredniego odpowiednika. Rozwiązania: (a) użyć zmiennych workspace-level w n8n, lub (b) dodać węzeł „Set” na początku workflow który ustawia wartości, dostępne potem jako `{{ $vars.roas_target }}`

Średnie

6. Dodanie pozycji węzłów na canvas (position)

Teraz: Węzły nie mają pozycji — kolejność jest liniowa przez pole `index`. Edytor Sembot rozkłada węzły sam.

Po zmianie: n8n wymaga współrzędnych `position: [x, y]` dla każdego węzła na canvas. Można wygenerować automatycznie (np. `[0, 0]`, `[220, 0]`, `[440, 0]`...) — to relatywnie prosta zmiana.

6

Co można zachować bez zmian



Dobra wiadomość — logika biznesowa pozostaje

Migracja do n8n to zmiana **reprezentacji** workflow (jak są zapisane), nie zmiana **tego co robią**. Wszystkie 38 szablonów — ich logika, warunki, integracje — zostają. Tylko "opakowanie" jest przepisywane na nowy standard.

Element	Działanie
✓ Sekcja meta (slug, name, category, icon, benefits)	Zachować — to dane Sembot-specyficzne, n8n ich nie dotknie
✓ Logika biznesowa każdego szablonu	Zachować — co workflow robi, pozostaje bez zmian
✓ Opisy i instrukcje konfiguracji	Zachować — pola name, description
✓ Status inactive na szablonach	Zachować — nowo importowane workflow są nieaktywne
✓ Flagi visible	Zachować — Sembot-specific, n8n nie potrzebuje

7

Podsumowanie — skala pracy i kolejne kroki

1

Doprecyzowanie wymagań

Przed jakąkolwiek pracą: spotkanie z szefem IT i potwierdzenie scenariusza. **Kompatybilność formatu** (opcja A) vs **n8n jako silnik** (opcja B) to fundamentalna różnica w zakresie projektu.

2

Zaprojektowanie mapowania typów

Stworzenie tabeli: każdy Sembot-specyficzny typ węzła (getter Google Ads, Phrase Monitor, create_task) musi mieć zdefiniowany odpowiednik w n8n lub decyzję o budowie custom node.

3

Budowa konwertera automatycznego

Napisanie skryptu/serwisu konwertującego stary format JSON na nowy. Przepisywanie 38 szablonów ręcznie jest możliwe, ale konwerter eliminuje błędy i przyspiesza pracę. Wyrażenia dynamiczne są najtrudniejszym elementem.

4

Custom nodes dla integracji Sembot

Zbudowanie własnych węzłów n8n dla: Google Ads getter, Microsoft Ads getter, Phrase Monitor getter, create_task action. To oddzielny projekt Node.js z własnym cyklem wdrożenia.

5

Testy i migracja szablonów

Każdy z 38 szablonów musi przejść testy: czy importuje się poprawnie do n8n, czy logika działa jak w v1. Szczególna uwaga na wyrażenia dynamiczne i warunki filtrów z globalnymi zmiennymi.



Szacunkowa skala zmian

38 szablonów do przepisania · **6 obszarów technicznych** wymagających zmian · **~4 custom nodes** do zbudowania od zera · Największe ryzyko: konwersja wyrażeń dynamicznych (każdy template ma ich kilkanaście) · Element możliwy do automatyzacji: generowanie UUID, pozycji, obiektu connections